

Mini-project, Part 1: The matrix-multiply unit

Building a big matrix multiply out of a small one

CS2.501 Advanced Computer Architecture · Monsoon 2026 · IIT Hyderabad

You will build a small matrix-multiply unit in BSV, use it to compute a large matrix product, and then explore three different ways of building it. The three designs all compute the same thing and all produce the same answers. They differ in how much they do per cycle, and therefore in how fast they run, how much hardware they cost, and how long their clock period has to be.

This is **Part 1** of the mini-project, and it is self-contained: it is specified, assessed and submitted on its own. In **Part 2** the unit you build here gets attached to a real RISC-V CPU as a custom instruction. §12 says what that involves and why this part ends where it does.

The specification fixes **what** your unit must do. It describes three designs you should build, because we want everyone measuring the same three things. What it does not do is tell you how they compare — that is the assignment, and the answer is not obvious in any of the three columns you will measure.

1 What you are building

A hardware unit that computes

$$C[4][4] += A[4][4] \times B[4][4]$$

with A and B holding signed 8-bit integers and C accumulating in signed 32-bit integers. All three matrices live *inside* the unit. Requests arrive through a queue and responses leave through another.

Wider accumulators than operands is not an accident, and §10 asks you to work out exactly how much wider they have to be.

Why the interface looks like this

Every request carries at most one 32-bit word in and returns at most one out. That is the width of a RISC-V register, and it is the single most important constraint in this project. A unit like this is meant to sit next to a CPU as a custom instruction, where the word coming in is `rs1`, the word going out is `rd`, and the small indices come from the immediate field.

So do not “improve” the interface by widening it. The width is the point, and §5 shows you what follows from it.

2 The unit and its contract

You implement:

```
interface MatMul_IFC;
  method Action          req (MM_Req r);
  method ActionValue #(MM_Rsp) rsp;
endinterface
```

Both methods are **guarded**: `req` blocks when you cannot accept a request, `rsp` blocks when you have no response ready. The harness relies on that back-pressure. Do not add explicit ready/valid signals.

There are four requests, defined in `MM_Types.bsv`, which you must not modify.

Request	Meaning
LoadA {row, chunk, word}	Write one 32-bit word into row <code>row</code> of A . The word carries four <code>Int#(8)</code> elements, element 0 in the low-order bits — the order a little-endian <code>lw</code> delivers them. At this shape a row of A is exactly 32 bits, so <code>chunk</code> is always 0 and one load fills a whole row.
LoadB {row, chunk, word}	The same, for B .
Mul	$C += A \times B$. Sixty-four multiply-accumulates.
ReadC {row, chunk}	Return one 32-bit slice of row <code>row</code> of C , and zero what it returns . At this shape one word is one element, so <code>chunk</code> selects the column and the whole of C takes sixteen reads.

Note the asymmetry: a row of A is one load, but a row of C is four reads. That is because the accumulator has to be wider than the operands, and it is not a wart — §10 asks what follows from it.

Two properties everything rests on

C **stays inside the unit and accumulates**. Each `Mul` adds another product on top of what is already there. Nothing clears C except reading it.

Every request returns exactly one response, in order. Only `ReadC` sets `has_value`; the others respond so that the response stream stays in lockstep with the request stream. Honour this even though nothing here obviously needs it.

3 What “correct” means

Your unit may do as much concurrently as you can make work, but the responses it produces must be identical to those of a unit that processed every request completely, one at a time, in order.

That single sentence is the entire correctness condition, and it is worth re-reading. It *permits* overlap — you are not required to finish one request before starting the next. It *forbids* any overlap that is observable.

The interesting question is which overlaps are observable. A design that is correct when each request takes one cycle can stop being correct when one of them takes several. Two of the three designs in §6 depend on getting this right.

Overflow: the workloads are built so that no accumulation overflows `Int#(32)`. If you overflow anyway, two’s-complement wrapping is what the reference does.

4 What the unit is for

A $4 \times 4 \times 4$ unit is not interesting by itself. It is interesting because you can build an arbitrarily large matrix multiply out of it. Suppose we want

$$C[2048][2048] += A[2048][2048] \times B[2048][2048]$$

Since $2048 = 4 \times 512$, this divides evenly into 4×4 tiles: 512 tiles across and 512 down. Each output tile is the sum of 512 small products, one for each 4-wide slice of the shared dimension:

```

for (i = 0; i < 2048; i += 4)           // 512 tile-rows of C
  for (j = 0; j < 2048; j += 4) {       // 512 tile-cols of C

    for (k = 0; k < 2048; k += 4) {     // 512 K-tiles: accumulate
      LoadA(0, &A[i+0][k]); LoadA(1, &A[i+1][k]);
      LoadA(2, &A[i+2][k]); LoadA(3, &A[i+3][k]);
      LoadB(0, &B[k+0][j]); LoadB(1, &B[k+1][j]);
      LoadB(2, &B[k+2][j]); LoadB(3, &B[k+3][j]);
      Mul();                             // C += A x B
    }

    for (r = 0; r < 4; r++)              // read the finished tile out
      for (c = 0; c < 4; c++)
        C[i+r][j+c] = ReadC(r, c);      // destructive: leaves C at zero
  }

```

Eight loads, one Mul, repeated. Then sixteen reads. That inner pattern is the whole workload, and everything in this project is about what it costs.

Two things follow immediately, and you should convince yourself of both.

Reading results out is nearly free. Sixteen reads against $512 \times 9 = 4608$ load-and-multiply requests — about 0.35% of the traffic. A deep enough accumulation amortises readout to nothing.

No reset request is needed. ReadC zeroes what it returns, so draining all sixteen elements leaves C clean and ready for the next output tile. The read *is* the reset. This is why the interface has no “clear” operation.

5 The cost model

This section is the one to understand before you write any BSV. There are two separate costs and they behave completely differently.

Instruction cost: fixed by the interface

One tile costs **9 requests** — eight loads and one Mul — and performs **64 multiply-accumulates**. So

$$\frac{64 \text{ MACs}}{9 \text{ requests}} \approx 7.1 \text{ MACs per request}$$

Eight of every nine requests move data, one 32-bit word at a time. The ninth is the only one that computes. **No hardware design you can build changes this number**, because it is a property of the interface and the tile shape, not of what happens inside the unit.

Cycle cost: set by your design

Requests arrive at most one per cycle. Let ℓ be the number of cycles a Mul occupies the arithmetic. Then how long a tile takes depends entirely on one question:

While a Mul is running, can the next tile's loads proceed?

Work out, for yourself, the cycles per tile in each case:

- a design in which the next tile's LoadA must wait for the running Mul to release A; and
- a design in which it need not.

You should get two different formulas in ℓ , and their ratio is the entire value of one of the three designs below. Do this on paper before you build anything — you will use it to predict your own results in §10.

The floor

Since the harness offers one request per cycle, no design can take fewer cycles than it is given requests:

$$\text{cycles} \geq \text{requests}$$

The harness reports `stalls = cycles - requests`: cycles in which your unit did not accept work. One unavoidable stall appears at the end of each workload, because the last response lands the cycle after the last request. Anything much above that is yours to account for.

6 Three designs

Build all three. They are genuinely different designs, and what makes them different is **what they compute in one cycle** — not variable names, and not a constant you tuned.

Design A — combinational

Mul performs the entire product in a single rule firing:

```
for (Integer i = 0; i < 4; i = i + 1)
  for (Integer j = 0; j < 4; j = j + 1) begin
    Acc acc = c[i][j];
    for (Integer l = 0; l < 4; l = l + 1)
      acc = acc + (signExtend (a[i][l]) * signExtend (b[l][j]));
    c_new[i][j] = acc;
  end
```

Those are `Integer` loops, so they fully unroll at elaboration: sixty-four multipliers and sixteen four-input adder trees, all in one combinational path. $\ell = 1$.

This is the design to build first. It is the simplest thing that is correct, and it is the reference the other two are checked against.

Design B — *K*-serial

Split the Mul across four cycles, computing one *outer product* per cycle. On cycle l , for all sixteen (i, j) at once:

$$C[i][j] += A[i][l] \times B[l][j]$$

That is sixteen multipliers and sixteen accumulating adders, working four times, instead of sixty-four multipliers and sixteen adder trees working once. ℓ is now greater than 1, and you will need a state machine and a way to tell an arriving request that the unit is busy.

This is where §3 stops being a formality. A **LoadA** that arrives while this **Mul** is mid-flight would change A underneath it. Work out precisely what the sequential-order requirement demands, and what it costs.

Design C — K -serial with double buffering

Design B, but with **two banks each of A and B** , used alternately. While a **Mul** reads one bank, the next tile's loads fill the other.

The extra state is small: two copies of A and B is 512 bits in total, plus a bit to say which bank is current. What it buys you is the difference between the two formulas you derived in §5.2.

Getting this correct is the interesting part. Think carefully about when the bank flips, and about what **ReadC** must do if a **Mul** is still in flight.

What to predict before you build

For each design, and *before* you measure, write down your prediction of:

- cycles per tile, as a formula in ℓ ;
- how the three will rank on cell count;
- how the three will rank on logic depth.

The third one is where people are most often wrong, and being wrong there in a way you can explain afterwards is worth more marks than being right by accident.

7 The two workloads

You cannot simulate a 2048×2048 multiply — it is over a billion requests. So the harness runs two smaller shapes that have the same structure. Both use the same unit and the same nine-request tile.

	W1	W2
shape ($M \times K \times N$)	$64 \times 64 \times 64$	$64 \times 512 \times 64$
output tiles	256	256
K -tiles per output tile	16	128
load requests	32,768	262,144
Mul requests	4,096	32,768
ReadC requests	4,096	4,096
total requests	40,960	299,008
multiply-accumulates	262,144	2,097,152
readout, as a share of requests	10%	1.4%

They differ in one thing only: how deep the accumulation runs before results are read out. W1 reads out every 16 **Muls**; W2 every 128. Report both. A design that looks good on one may not look good on the other, and noticing why is a large part of what this project is for.

8 What to measure

Cycle count alone is a bad measure of a hardware design, because it says nothing about **how long a cycle has to be**. A unit that does everything in one enormous combinational sweep wins on cycles while being unbuildable at any sensible clock rate — and Bluesim will never tell you, because it counts cycles and knows nothing about time.

So you report numbers from two tools:

Measured	From
W1 cycles, W2 cycles, stalls	<code>make</code>
<code>cells</code> , <code>logic_depth</code>	<code>make depth</code>

`logic_depth` is the number of gate levels on the longest combinational path — an estimate of what sets your clock period. `cells` is a rough area count.

`logic_depth` is a proxy, not static timing analysis. It counts gate levels and ignores gate and wire delay. It is deterministic and reproducible, which is what makes it useful for comparing *your own* designs against each other. Do not quote it in nanoseconds.

Then compute, for each design:

Derived	What it estimates
<code>cycles</code> $\times$ <code>logic_depth</code>	time to finish the job — what a user cares about
<code>cells</code> $\times$ <code>logic_depth</code>	cost of the hardware — roughly, area $\times$ how slow it is
MACs per request	arithmetic delivered per unit of interface bandwidth

There is no single winner, and that is the assignment

Build the table, then sort it by each column in turn. If the ranking changes between columns, you have found the trade-off and you can argue about it.

Your job is not to find “the best” design. It is to **map the trade-off with your own measurements and then argue for a choice**, given a stated priority. State the priority you are optimising for, and defend the choice against the columns that disagree with you.

9 Building and measuring

What you are given:

<code>MatMul.bsv</code>	your file — ships as a naive design; edit it, replace it, rewrite it
<code>MM_Types.bsv</code>	the contract of §2. Do not modify
<code>TbMatMul.bsv</code>	the testbench, and the same one we grade with. Do not modify
<code>Makefile</code> , <code>area-delay.sh</code>	the two measurements of §8

```
export PATH=$HOME/tools/bsc-2026.01-debian-11.11/bin:$PATH
```

```
make          # compile, run W1 and W2, check correctness, report cycles
make depth    # cells and logic_depth of mkMatMul
make schedule # bsc's rule-conflict report -- see section 10
make clean
```

`make depth` needs a one-time install, which requires no root:

```
pip install --user 'yowasp-yosys==0.68.0.0.post1208'
```

Correctness is checked first and it is absolute. A fast wrong answer scores nothing. This is not a formality — the most common way to lose correctness here is by successfully making something faster.

One environment note that will otherwise cost you an hour: `yowasp-yosys` runs in a sandbox that can only read files **at or below the current directory**. If you point it at `/tmp` you will get “file not found” for a file that plainly exists.

10 What to work out

These are not optional extras. They are the project, and they are what the report is marked on.

Did your prediction hold? You derived cycles-per-tile as a formula in ℓ in §5.2, and predicted W1 and W2 cycles for all three designs in §6.4. Compare against what you measured. Where the formula and the measurement disagree, find out why — the discrepancy is usually one or two cycles per tile and it always has a specific cause.

Where do your cycles go? If a design takes substantially more cycles than it was given requests, something is stalling. What, and why? Count before you optimise.

Why is MACs-per-request the same for all three designs? All three should give you an identical number here. Explain why that is, and say what would have to change to move it. This is the most important question in the project.

What sets your longest path? Look at what `Mul` computes as a single expression and count the operations in series along one output. Which term dominates — the multiply, or the summation over K ? Then be careful, because the obvious move is a trap: “fewer multipliers” and “shorter critical path” are different properties and they do not always move together.

What happens when a `LoadA` arrives while a `Mul` is in progress? Work out precisely what §3 demands here for designs B and C. There is more than one legitimate answer and they cost different amounts.

Why does W2 cost more than W1 per unit of readout, but less per MAC? Put numbers on both. Then: which of your three designs is most sensitive to the difference between the two workloads, and why?

How wide does the accumulator have to be? Derive a bound on W_ACC in terms of W_ELEM , K , and the number of `Muls` between readouts. Then check: at $W_ACC = 16$, how many `Muls` could you safely accumulate before reading out? This is why the readout structure exists.

When does a DMA-fed unit stop being instruction-bound? Appendix A describes the change real accelerators make to escape the limit of §5.1. Read it, then answer two things. First: at what tile size does a DMA-fed $4 \times 4 \times 4$ unit become limited by memory bandwidth rather than by instruction supply? Put a number on the bandwidth it would need. Second: name one guarantee the operating system has to provide that it does not have to provide for the interface in §2.

When BSV does not do what you expect. If two rules that you believe should run in the same cycle never do, `bsc` has already told you why — you just have not looked. Run `make schedule` and read the conflict report. It names the two rules and the exact resource they contend for. `bsc` schedules **whole rules**, and that fact has consequences for how you structure them.

11 Handing in

- `MatMul.bsv` implementing `mkMatMul` — your **chosen** design, which must be correct on both workloads.
- Your other two designs, as `MatMulA.bsv`, `MatMulB.bsv`, `MatMulC.bsv` or similar. All three must compile and run.
- `MM_Types.bsv` and `TbMatMul.bsv`, unmodified.
- `REPORT.md`, **three pages maximum**, containing:
 - the measurement table — three designs $\times$ five measured columns $\times$ three derived ones;
 - your predictions from §6.4, and where they were wrong;
 - your answers to §10;
 - which design you chose, the priority you chose it under, and the strongest argument *against* your choice.

Three pages is a limit, not a target. A tight two-page report that answers §10 scores better than five pages that do not.

12 Where this goes: Part 2

Part 1 stops exactly where the interesting question starts.

§5.1 establishes that eight of every nine requests move data, and that no design you build changes it. But that measurement is taken with a testbench that offers a request every cycle, unconditionally, for free. A real CPU has to *earn* every one: fetch and decode an instruction for it, execute the load that supplies its data, and pay for the loop around them.

Part 2 attaches the unit you build here to Fife — the pipelined RV32I CPU you booted in lab0 — as a custom instruction, and measures what a real single-issue in-order pipeline does to the number in §5.1. Among the things that turn up there:

- a unit that takes several cycles interacts with the CPU’s hazard logic in a way a single-cycle one never does;
- retirement is in-order, so a slow `Mul` holds up completed work queued behind it;
- and the largest speedup available may be in how the loop is scheduled rather than in anything you change in the hardware.

The unit you submit for Part 1 is the one that gets attached. That is why `MM_Types.bsv` is fixed, and why the interface is exactly one 32-bit word in and one out: each request becomes a single RISC-V instruction, with the word in as `rs1`, the word out as `rd`, and `row` and `chunk` coming from the immediate field. Nothing you build here gets thrown away.

Part 2’s specification is announced later in the semester. Nothing in it requires you to have chosen any particular design in Part 1 — only to have one that is correct.

Appendix A — Escaping the register file: DMA

This appendix is background, not a build task. Nothing here is graded except the two questions in §10 that point at it. It is here because the project would otherwise end on a limitation without saying how the limitation is actually escaped.

The problem, restated

Eight of every nine requests move data, one 32-bit word at a time. That is not a weakness of your design — §5.1 shows no design can change it — and it is not a weakness of the interface either. It follows from one fact: **a request is an instruction, and an instruction can name one register**. Thirty-two bits per instruction is the ceiling, and everything else in this project happens underneath it.

The fix: stop moving data through the register file

Give the unit its own connection to memory, and change what the instruction carries. Instead of sending the *data*, the CPU sends an *address and a shape*:

```
LoadTileA (base_address, row_stride)  // fetches all 4 rows itself
LoadTileB (base_address, row_stride)
Mul      ()
```

The nine requests per tile become three, and MACs per instruction goes from 7.1 to about 21. But the real gain is not there. Once the unit can generate its own addresses, one instruction can describe the *entire K-loop* — base addresses, strides, and a count — and the CPU issues a handful of instructions per output tile instead of thousands. This is called a **descriptor**, and it is how every production accelerator is fed.

What the hardware needs

- A **bus master port** — AXI4 or TileLink. Your unit currently has only a slave interface: it answers requests. Now it has to *make* them.
- An **address generator**: base, strides, counts, producing a sequence of burst addresses without CPU help.
- **Outstanding-transaction tracking**. Memory answers late, and out of order. The unit must know which returning data belongs to which row.
- A **buffer to absorb memory latency** — and note what this is: double buffering again, exactly as in Design C, but now hiding DRAM latency instead of Mul latency. The same structural idea reappears one level up.
- **Completion signalling**, so software can know the transfer is done.

What the software and the OS need — the expensive part

This is where DMA stops being a hardware problem. All of the following are required, and none of them apply to the interface in §2:

- **Cache coherence**. The DMA engine reads memory that the CPU may be holding dirty in its cache. Either the port participates in the coherence protocol, or software must flush and invalidate around every transfer — and get it right every time.
- **Address translation**. Does the unit see virtual or physical addresses? Virtual means an IOMMU or a shared TLB. Physical means the OS must hand over physical addresses, which user code does not have.
- **Page pinning**. The OS must guarantee the pages do not move or get swapped out while the transfer is in flight.

- **Asynchronous completion.** Polling a status register, an interrupt, or a fence-like instruction that blocks. The programming model is no longer sequential, and this is where the bugs live.

A useful way to see the trade: the interface in §2 is slow but needs *nothing* from the operating system, because every byte moves through a register the CPU already owns. DMA is fast and needs a great deal.

What you actually bought

The bottleneck moves. It does not vanish.

A $4 \times 4 \times 4$ tile reads 16 bytes of A and 16 bytes of B and performs 64 multiply-accumulates. That is **2 MACs per byte**. If the array does 64 MACs per cycle at 1 GHz, feeding it requires 32 GB/s — serious DRAM bandwidth for an array this small. **A DMA-fed $4 \times 4 \times 4$ unit becomes memory-bound almost immediately.**

Both ratios improve with tile size, and they improve the same way. For an $S \times S \times S$ tile:

$$\text{MACs per byte} = \frac{S^3}{2S^2} = \frac{S}{2} \qquad \text{MACs per instruction} \approx \frac{S^3}{S^2/2} = 2S$$

Both are linear in the tile edge, and they differ by exactly a factor of four — the number of `Int#(8)` elements in a 32-bit word. **DMA does not reduce how much data must move. It reduces how many instructions it takes to name it.**

So the argument for large tiles arrives twice, from opposite directions: from the instruction side, because instruction bandwidth is scarce; and from the memory side, because DRAM bandwidth is scarce. They are the same argument about the same thing — reuse — and the second one is the roofline model you will meet later in the course.

Where you have already seen this

Every serious accelerator is built this way. A TPU keeps a large on-chip buffer filled by descriptor-driven DMA rather than by instructions carrying data. Modern GPUs have dedicated tile-fetch hardware that copies a strided multidimensional block from memory into shared memory from a descriptor, so that one instruction moves a whole tile. The pattern is identical: the compute unit names data by address and shape, never by value.

Which is why the interface in §2 is worth understanding precisely. It is the simplest thing that works, it costs the operating system nothing, and it shows you exactly where the ceiling is — which is the only way the reason for the next design is ever visible.